# Laboratory Manual

## Course: Algorithm Analysis and Design Lab

### Experiment Title: All-Pairs Shortest Path Using Recursion and Dynamic Programming

# 1. Experiment Information

| Item | Details |
|---|---|
| **Course Title** | Algorithm Analysis and Design Lab |
| **Experiment No.** | 2 |
| **Experiment Title** | Recursive and Dynamic Programming-Based All-Pairs Shortest Path |
| **Programming Language** | C |
| **Algorithm Technique** | Dynamic Programming |
| **Recommended IDE** | Code::Blocks / Dev-C++ / Visual Studio Code |
| **Submission Type** | Source Code + Report + Output Screenshot |

# 2. Objective of the Experiment

The objectives of this laboratory experiment are:

- Understand the concept of Dynamic Programming (DP).
- Identify overlapping subproblems in recursive solutions.
- Implement recursive shortest path calculation.
- Optimize recursive computation using Dynamic Programming.
- Compare recursive and DP approaches..

# 3. Problem Description

Develop a program to find the **shortest path between all pairs of vertices** in a weighted graph. Students must:

- ☐ First implement the solution using **pure recursion**.
- ☐ Identify repeated computations.
- ☐ Apply **Dynamic Programming** to optimize the solution.

# 4. Concepts Covered

| Concept | Description |
|---|---|
| **Recursion** | Solving problems using self-calling functions |
| **Overlapping Subproblems** | Same subproblems solved repeatedly |
| **Memoization** | Storing previously computed results |
| **Dynamic Programming** | Optimized recursive computation |
| **Graph Algorithms** | Shortest path calculation |

# 5. Input Requirements

- Number of vertices
- Weighted adjacency matrix

Example:

0 3 INF 7
8 0 2 INF
5 INF 0 1
2 INF INF 0

Where:

INF = No direct edge

# 6. Recursive Approach

## Input Requirements

Students will initially solve the shortest path problem recursively.

**Recursive Formula**

```
Shortest(i,j,k) =
min(
    Shortest(i,j,k-1),
    Shortest(i,k,k-1) + Shortest(k,j,k-1)
)
```

# 7. Dynamic Programming Optimization

Dynamic Programming stores previously computed values.

Students will use:

```
dp[i][j]
```

to avoid recalculating shortest paths.

# 8. Dynamic Programming Strategy

| Step | Description |
|---|---|
| 1 | Initialize distance matrix |
| 2 | Use existing edges as base values |
| 3 | Update paths using intermediate vertices |
| 4 | Store computed shortest distances |

# 9. Sample Pseudocode

## Recursive Version

```
Shortest(i,j,k)
if(k == 0)
    return graph[i][j]
return min(
    Shortest(i,j,k-1),
    Shortest(i,k,k-1) + Shortest(k,j,k-1)
)
```

## Dynamic Programming Version

```
for(k=0;k<n;k++)
{
    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
            if(dist[i][k] + dist[k][j] < dist[i][j])
            {
                dist[i][j] = dist[i][k] + dist[k][j];
            }
        }
    }
}
```

# 10. Expected Learning Outcomes

After completing this experiment, students will be able to:

- Understand Dynamic Programming principles
- Detect overlapping subproblems
- Optimize recursive algorithms
- Implement Floyd-Warshall Algorithm
- Compare recursive and DP performance

# 11. Report Contents

Students must include:

(a) Objective
(b) Theory
(c) Recursive Algorithm
(d) Dynamic Programming Algorithm
(e) Source Code
(f) Input/Output
(g) Complexity Analysis
(h) Comparison Discussion
(i) Conclusion